Module – Linux services theory

1. Install the Apache web server on your Linux machine using the package manager (apt or yum), and confirm the installation by running the command to check its version.

   ANS :
   Step 1 :
   - Ubuntu/Debian (APT)
     - sudo apt update
       sudo apt install apache2 -y

   step 2 :
   - Check Apache version:
     - apache2 -v
   - Example Output:
     - Server version: Apache/2.4.58 (Ubuntu)

   Step 3 :
   - RHEL/CentOS
     - sudo yum install httpd -y

     or
     - sudo dnf install httpd -y
   - ➢ Check version:
   - httpd -v

2. Start the Apache service using systemctl, then use systemctl status to verify it is running.

   ANS :

   step 1 :
   - **Ubuntu/Debian**
     - sudo systemctl start apache2
       sudo systemctl status apache2

   step 2 :
   - **RHEL/CentOS**
     - sudo systemctl start httpd
       sudo systemctl status httpd
   - ➢ Expected status:
   - Active: active (running)

3. Stop and restart the Apache service using systemctl, and observe the difference between stop, start, and restart commands.

Hint: Use 'systemctl stop apache2', 'systemctl start apache2', and 'systemctl restart apache2' (or 'httpd' on some systems).

ANS :

1) Stop Service:
   - sudo systemctl stop apache2
2) Start Service:
   - sudo systemctl start apache2
3) Restart Service:
   - sudo systemctl restart apache2

➢ Difference Between Commands

stop	Stops the Apache service completely
start	Starts Apache if it is stopped
restart	Stops and immediately starts Apache again

4. Open your browser and access http://localhost or your Linux machine's IP address to verify that the Apache default web page is displayed.

ANS :

step 1 :

- Find your IP address:
  - ip addr

step 2 :

- Open a browser and visit:
  - http://localhost

➢ You should see:

- Testing 123 - Apache HTTP Server

➢ This confirms Apache is running successfully.

5. Write a short comparison (3-4 lines) between Apache and Nginx web servers, mentioning one key use-case for each based on your research.
   ANS :

Apache	Nginx
Process-based architecture	Event-driven architecture
Better support for .htaccess files	Higher performance for static content
Easier configuration for beginners	Handles many concurrent connections efficiently
Commonly used for PHP-based websites	Commonly used as a reverse proxy and load balancer

- **Apache:** Best for shared hosting and PHP applications such as **WordPress**.
- **Nginx:** Best for high-traffic websites, reverse proxy servers, and load balancing.